Welile Wallet Drift & User Negatives

Source-of-Truth Map & Root-Cause Documentation

Generated 2026-05-07 · Ticket #DEBT-001 follow-up · Classification: Internal / CFO & CTO

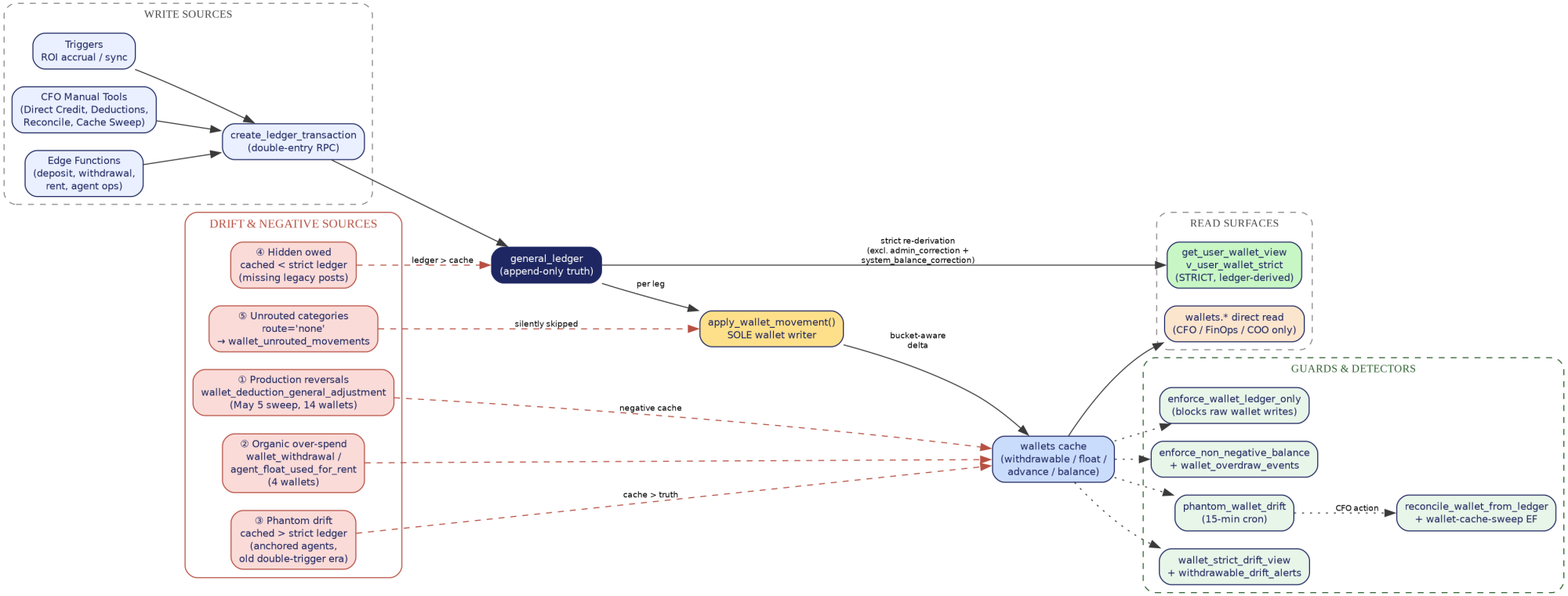

Fig. 1 — End-to-end wallet write/read path. Red dashed arrows mark the five drift sources; green dashed boxes are the guards and detectors that catch them.

1. The Single Source of Truth

Every UGX moved on the Welile platform is recorded as a balanced double-entry pair in **general_ledger**. That table is append-only and is the only legally authoritative record of money. Every other balance the platform shows — the **wallets** cache, the agent dashboard pill, the supporter wallet card — is a *derived* view of the ledger. When derivation drifts away from the ledger, users see numbers that do not exist, or fail to see numbers that do. That gap is what we call **drift**.

Two read surfaces, very different contracts

Strict (end-user) surface: `get_user_wallet_view()` and `v_user_wallet_strict` re-derive every bucket live from the ledger, exclude paired admin corrections, honour fresh-start anchors and pending withdrawal holds, and clamp at zero. This is what tenants, agents, supporters and landlords see — and what the withdrawal gate enforces.

Cached (operator) surface: the `wallets` table holds `balance / withdrawable_balance / float_balance / advance_balance`, maintained by `apply_wallet_movement()`. Operator dashboards (CFO, FinOps, COO, CEO, CTO, HR, Manager, Executive) read it raw — because their job is to *see* the drift and fix it.

2. The Sole Writer Rule

Since 2026-04-23 only one function is permitted to mutate wallet bucket fields: `apply_wallet_movement(user_id, category, amount, direction)`. It is invoked by `create_ledger_transaction` for every ledger leg, after the leg is already persisted. The legacy `sync_wallet_from_ledger()` trigger has been neutered to `RETURN NEW` — it stays attached for backward compatibility but performs no work. The DB trigger `enforce_wallet_ledger_only` rejects any other UPDATE to the wallet bucket fields unless the session sets the `wallet.sync_authorized` flag (the only legal cache-correction path).

3. The Five Sources of Drift & Negatives

#	Source	Where it shows	Mechanism	Today's exposure
①	**Production reversals** `wallet_deduction_general_adjustment`	Negative cached withdrawable	May-5 cleanup posted balanced production legs that removed money already shown to users; cache went red because there was nothing to debit against.	14 wallets ~UGX 51.7M
②	**Organic over-spend** `wallet_withdrawal`, `agent_float_used_for_rent`, `agent_repayment`	Negative cached withdrawable or float	Withdrawal or float-allocation succeeded against a stale cache snapshot, then the strict re-derivation revealed the real position was lower.	4 wallets
③	**Phantom drift** cached > strict	Inflated headline, blocked withdrawals	Pre-2026-04-23 era: both `sync_wallet_from_ledger` AND `apply_wallet_movement` updated balance per leg, double-counting every credit. Anchored agents still carry that residue.	~UGX 54M (anchor backfill)
④	**Hidden owed** cached < strict	User missing money they earned	Legacy ledger posts that pre-date the cache, or leg pairs whose wallet leg failed to fire. Strict view sees them; cache does not.	~UGX 78M
⑤	**Unrouted categories** route='none'	Silent zero-impact	`apply_wallet_movement` now logs every category with no bucket route into `wallet_unrouted_movements` instead of silently returning. Two known offenders: `test_funds_cleanup`, `proxy_investment_commission`.	Tracked daily

All five sources converge on the same fix path: post a **balanced** `system_balance_correction` pair under `classification='admin_correction'`, then sync the cache via the `wallet.sync_authorized` session flag. End-user views filter that exact (admin_correction, system_balance_correction) tuple out, so the user never sees the correction; operator dashboards do.

4. Guards & Detectors

Layer	Object	What it does
DB trigger	`enforce_wallet_ledger_only`	Rejects any UPDATE to `wallets.balance / withdrawable_balance / float_balance / advance_balance` unless the `wallet.sync_authorized` session flag is set. Closes the door on direct cache writes.
DB trigger	`enforce_non_negative_balance`	Floors any bucket update at 0 and logs the clamp into `wallet_overdraw_events` (CFO / COO / Operations read). Every previously-silent overdraw is now attributable.
DB trigger	`trg_enforce_production_april_cutoff`	Forces every `general_ledger` row dated ≥ 2026-04-01 to classification='production' (only admin_correction allowed distinct). Stops new historical data from polluting the production net.
Cron (15 min)	`detect_phantom_wallet_drift` → `phantom_wallet_drift`	Continuous scan of cache vs strict ledger. Surfaces in CFO → Reconciliation → PhantomDriftPanel.
Cron (15 min)	`detect_withdrawable_drift_alerts` → `wallet_withdrawable_drift_alerts`	CFO-configurable UGX threshold; emits `wallet.drift_alert.raised` system events for high / critical drift.
View	`wallet_strict_drift_view`, `wallet_anchored_drift_view`, `wallet_ledger_truth_view`	Three diagnostic lenses: strict-vs-cache, anchored agents, and all-time double-entry truth.
RPC	`reconcile_wallet_from_ledger(user_id, reason)`	CFO-only, ≥10-char reason. Posts a balanced admin_correction pair and aligns the cache. Audited to `audit_logs` + `system_events`.
Edge fn	`wallet-cache-sweep`	Hard-caps deduction at (cached – strict) so it can never touch real customer-owed money. Surfaced in CFO → Reconciliation → CacheSweepPanel.
RPC	`begin_ledger_maintenance / end_ledger_maintenance`	Time-boxed window (≤240 min, ≥10-char reason) that lets the CFO bypass the RPC-only guard for raw corrections. Driven from the new Ledger Maintenance card on the CFO Overview.

5. Read-Path Discipline

The single most common cause of user-facing negatives in 2026-Q2 was components reading `wallets.balance` directly. The rule is now enforced both in memory and at build time:

User-facing wallet UIs (paths under `src/components/wallet`, `/payments`, `/agent`, `/supporter`, `/tenant`, `/landlord`, `/dashboards` and `src/pages/{agent,supporter,tenant,landlord}`) MUST call `get_user_wallet_view` via `useAvailableBalance` / `useAgentBalances` / `computeLedgerAvailable`. Operator paths (`cfo`, `financial-ops`, `manager`, `executive`, `ceo`, `coo`, `cto`, `hr`, `crm`, `cmo`, `reconcile`) are explicitly allowlisted to read the cache. `scripts/guard-frontend-ledger-writes.mjs` enforces this on every build.

6. Today's Picture (2026-05-07)

Metric	Value	Notes
Wallets with negative cached buckets	0	Wiped via `wallet_deposit` reseed under ticket #DEBT-001.
User-visible negatives (strict view)	18	14 from the May-5 production reversal sweep; 4 organic withdrawals / float.
Total visible negative exposure	UGX –74.29M	Down from UGX 178.9M cache↔ledger gap at the 2026-04-30 baseline.
Phantom drift (cached > strict)	Tracked nightly	Anchor-window agents reviewed via HistoricalDriftReviewPanel.
Hidden owed (cached < strict)	Tracked nightly	Released explicitly via `release_historical_drift` RPC (CFO-only).

7. One-line Summary

Drift = (wallets cache) – **(general_ledger strict re-derivation).** Positive drift is phantom money the user can see but not withdraw; negative drift is owed money the user cannot see. Both are healed only by posting a balanced admin_correction pair and re-syncing the cache through the sole writer. Everything else in this document is mechanism.